The University of Faisalabad

Computer Organization & Assembly Language

Course: Computer Organization & Assembly Language (Lab) - CS-318

Comparison and Conditions

Aim:

Compare two numbers and display which one is greater, smaller, or equal.

Theory:

- In assembly, the CMP instruction is used to compare two registers or values.
- Based on the comparison, **conditional jumps** like JG, JL, or JE decide the program flow.
- Labels mark the positions where control will jump after a condition is met.

```
org 100h


START:              ; ---- Program Start ----
    mov al, 5       ; first number
    mov bl, 3       ; second number


    cmp al, bl      ; compare AL and BL
    jg GREATER      ; jump if AL > BL
    jl SMALLER      ; jump if AL < BL
```

```
EQUAL:              ; label for equal condition
    mov dx, offset MSG_EQ
    mov ah, 9
    int 21h
    jmp EXIT        ; go to exit

GREATER:            ; label for greater condition
    mov dx, offset MSG_GT
    mov ah, 9
    int 21h
    jmp EXIT

SMALLER:            ; label for smaller condition
    mov dx, offset MSG_LT
    mov ah, 9
    int 21h

EXIT:               ; ---- Program End ----
    mov ah, 4Ch
    int 21h

MSG_GT db 'AL is Greater$'
```

```
MSG_LT db 'AL is Smaller$'

MSG_EQ db 'Both are Equal$'
```

Label Explanation:

- START: → Program begins.
- GREATER: → Executes when AL > BL.
- SMALLER: → Executes when AL < BL.
- EQUAL: → Executes when AL = BL.
- EXIT: → Ends the program.

Conditional Jumps

Aim:

Check if number is even or odd using JE and JMP.

Theory:

- DIV divides AL by a number and stores remainder in AH.
- If remainder = 0 → even number, else → odd.
- JE (Jump if Equal) is used to branch to the even block when the remainder is zero.

```
org 100h

START:

    mov al, 6

    mov ah, 0       ; clear AH before division

    mov bl, 2

    div bl          ; AL = quotient, AH = remainder
```

```
    cmp ah, 0
    je EVEN         ; jump if remainder = 0


ODD:
    mov dx, offset MSG_ODD
    mov ah, 9
    int 21h
    jmp EXIT


EVEN:
    mov dx, offset MSG_EVEN
    mov ah, 9
    int 21h
EXIT:
    mov ah, 4Ch
    int 21h
MSG_EVEN db 'Number is EVEN$'
MSG_ODD  db 'Number is ODD$'
```

Label Explanation:

- START: → Program begins.
- EVEN: → Executes if remainder = 0 (JE condition).
- ODD: → Executes if remainder ≠ 0.
- EXIT: → Ends the program.

Unconditional Jump

Aim:

Display * continuously using JMP (no condition).

Theory:

JMP is an **unconditional jump** — it always transfers control to the target label.
This causes the program to repeat endlessly, creating an infinite loop.

```
org 100h


START:
    mov ah, 2
    mov dl, '*'
    int 21h
    jmp START      ; jump to START again (infinite loop)
```

Label Explanation:

- START: → The loop start point.
- JMP START → Goes back to the same point repeatedly.

Relative Addressing

Aim:

Demonstrate short forward and backward jumps.

Theory:

Relative addressing means the jump target is **nearby** (within ±128 bytes).
These jumps move control forward or backward relative to current location.

```
org 100h

BEGIN:
    mov ah, 2
    mov dl, 'A'
    int 21h
    jmp SKIP        ; forward jump (relative jump ahead)

SHOW_B:
    mov dl, 'B'     ; this will be skipped
    int 21h

SKIP:
    mov dl, 'C'
    int 21h
    jmp BACK        ; backward jump (relative)

BACK:
    jmp BACK        ; infinite loop for demonstration
```

Label Explanation:

- BEGIN: → Program start.
- SKIP: → Target for forward jump.
- BACK: → Backward jump (loop demonstration).
- SHOW_B: → This part is skipped due to jump.

Types of Jumps

Aim:

Show multiple jump types (JE, JNE, JG, JL).

Theory:

Different jumps are used for conditions:

- JE → Jump if equal
- JNE → Jump if not equal
- JG → Jump if greater
- JL → Jump if less

```
org 100h


START:
    mov al, 4
    mov bl, 6


    cmp al, bl
    je EQUAL
    jne NOTEQUAL
```

```
    jg GREATER
    jl SMALLER

EQUAL:
    mov dx, offset MSG_EQ
    mov ah, 9
    int 21h
    jmp EXIT

NOTEQUAL:
    mov dx, offset MSG_NE
    mov ah, 9
    int 21h
    jmp EXIT

GREATER:
    mov dx, offset MSG_GT
    mov ah, 9
    int 21h
    jmp EXIT

SMALLER:
    mov dx, offset MSG_LT
```

```
    mov ah, 9
    int 21h

EXIT:
    mov ah, 4Ch
    int 21h

MSG_EQ db 'Equal$'
MSG_NE db 'Not Equal$'
MSG_GT db 'Greater$'
MSG_LT db 'Smaller$'
```

Label Explanation:

- EQUAL: → Executes if AL = BL.
- NOTEQUAL: → Executes if AL ≠ BL.
- GREATER: → Executes if AL > BL.
- SMALLER: → Executes if AL < BL.
- EXIT: → Ends program.

Sorting Example (Bubble Sort — 3 Numbers)

Aim:

Sort 3 numbers in ascending order using loops and jumps.

Theory:

Bubble sort repeatedly compares adjacent values and swaps them if needed.
Here, CMP, JBE, and LOOP are used to control iterations.

```
org 100h

DATA_START:
    nums db 5, 3, 1      ; unsorted numbers

START:
    mov cx, 2            ; number of passes
OUTER_LOOP:
    mov si, offset nums
    mov bx, 2            ; number of comparisons per pass

INNER_LOOP:
    mov al, [si]
    mov dl, [si+1]
    cmp al, dl
    jbe NO_SWAP          ; jump if already in order
    mov [si], dl         ; swap
    mov [si+1], al
NO_SWAP:
    inc si
    dec bx
    jnz INNER_LOOP       ; inner loop continue
    loop OUTER_LOOP      ; outer loop continue
```

```
DISPLAY:
    mov si, offset nums
    mov cx, 3
PRINT_LOOP:
    mov dl, [si]
    add dl, 30h
    mov ah, 2
    int 21h
    inc si
    loop PRINT_LOOP

EXIT:
    mov ah, 4Ch
    int 21h
```

Label Explanation:

- DATA_START: → Start of data (array).
- OUTER_LOOP: → Outer loop for number of passes.
- INNER_LOOP: → Compares and swaps numbers.
- NO_SWAP: → Skip if no swap needed.
- DISPLAY: → Prints sorted numbers.
- EXIT: → Program end.

Topic	Concept	Key Jumps Used

Topic	Concept	Key Jumps Used
Comparison	Compare values	JG, JL
Conditional Jump	Even/Odd check	JE, JMP
Unconditional Jump	Infinite loop	JMP
Relative Addressing	Short jump demo	Forward/Backward
Types of Jump	Multiple examples	JE, JNE, JG, JL
Sorting	Loop + jump combo	JBE, JNZ, LOOP
